Data Structures (CSN12101)
ASSIGNMENT-3

Name : Aayansh Niranjan Date : 21/01/2026
Reg_No.: 20253004

Q1. An e- commerce company receives a large number of customer orders every day. Each order is identified by a unique Order ID. Before sending orders to the warehouse for packing, the system must arrange all Order IDs in ascending order so that the packing and delivery process becomes easier. Write a C program to implement Merge Sort to sort a large-sized array of Order IDs and also find the total number of comparisons performed during sorting and the time required to sort the array.

A1. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long comparisons=0;

void merge(int a[],int l,int m,int r){
    int i=l,j=m+1,k=0;
    int n=r-l+1;
    int *t=malloc(n*sizeof(int));
    while(i<=m&&j<=r){
        comparisons++;
        if(a[i]<=a[j]) t[k++]=a[i++];
        else t[k++]=a[j++];
    }
    while(i<=m) t[k++]=a[i++];
    while(j<=r) t[k++]=a[j++];
    for(i=0;i<n;i++) a[l+i]=t[i];
    free(t);
}

void mergeSort(int a[],int l,int r){
    if(l<r){
        int m=(l+r)/2;
        mergeSort(a,l,m);
        mergeSort(a,m+1,r);
        merge(a,l,m,r);
    }
}

int main(){
    int n;
    scanf("%d",&n);
    int *a=malloc(n*sizeof(int));
```

```
for(int i=0;i<n;i++) scanf("%d",&a[i]);
    clock_t s=clock();
    mergeSort(a,0,n-1);
    clock_t e=clock();
    for(int i=0;i<n;i++) printf("%d ",a[i]);
    printf("\n");
    printf("Number of comparisons :  %lld\n",comparisons);
    printf("Total time taken : %f\n",(double)(e-s)/CLOCKS_PER_SEC);
    free(a);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_3$ ./p1
20
53 76 26 83 21 45 52 83 28 945 24 68 214 57 43 98 822 56 87 76
21 24 26 28 43 45 52 53 56 57 68 76 76 83 83 87 98 214 822 945
Number of comparisons :  64
Total time taken : 0.000013
```

Q2. Write a C program to implement Quick sort in a given array of larger size.
• You have to find the number of comparisons and time required to sort the array.
• What if array elements are already arranged in sorted order? How will you this problem?

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long comparisons=0;

int partition(int a[],int l,int r){
    int p=a[r];
    int i=l-1;
    for(int j=l;j<r;j++){
        comparisons++;
        if(a[j]<=p){
            i++;
            int t=a[i];
            a[i]=a[j];
            a[j]=t;
        }
    }
    int t=a[i+1];
    a[i+1]=a[r];
    a[r]=t;
    return i+1;
}

void quickSort(int a[],int l,int r){
    if(l<r){
        int p=partition(a,l,r);
        quickSort(a,l,p-1);
        quickSort(a,p+1,r);
    }
}

int main(){
    int n;
    scanf("%d",&n);
    int *a=malloc(n*sizeof(int));
    for(int i=0;i<n;i++) scanf("%d",&a[i]);
    clock_t s=clock();
    quickSort(a,0,n-1);
    clock_t e=clock();
    for(int i=0;i<n;i++) printf("%d ",a[i]);
    printf("\n");
    printf("Number of comparisons : %lld\n",comparisons);
    printf("Total time taken : %f\n",(double)(e-s)/CLOCKS_PER_SEC);
    free(a);
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_3$ ./p2
20
53 76 26 83 21 45 52 83 28 945 24 68 214 57 43 98 822 56 87 76
21 24 26 28 43 45 52 53 56 57 68 76 76 83 83 87 98 214 822 945
Number of comparisons : 56
Total time taken : 0.000006
```